



北京師範大學
BEIJING NORMAL UNIVERSITY

计算机图形学 实验报告

姓名 杨博文 学号 202311081015
院系 人工智能学院 专业 计算机科学与技术

2025 年 6 月 27 日

摘 要

计算机图形学的第九次实验，主要内容包括 SMPL 人体模型。

目录

1 运行环境及运行方式	3
2 任务 1:demo 输出与 $body_{pose}$	3
2.1 任务要求	3
2.2 代码修改	3
2.3 运行结果	4
2.4 $body_{pose}$ 含义	5
2.5 不同初始化值	5
2.6 与可微渲染	6
3 任务 2:Blender 调整	6
3.1 任务要求	6
3.2 运行结果	7
3.3 蒙皮的思想	8
3.4 蒙皮公式	8
3.5 合理限制	8
4 任务 3:VPoser	8
4.1 任务要求	8
4.2 代码修改与结果展示	9
4.2.1 demo	9
4.2.2 VPoser	10
4.3 输入输出与作用	11
4.4 深度学习网络	11
4.5 进阶使用	11
5 结束语	12

1 运行环境及运行方式

硬件为 Macbook Air, M2 芯片, 操作系统为 MacOS Sequoia 15.0。

IDE 为 Pycharm2024.1.3。虚拟环境由 conda 管理。

使用 Blender, version 4.4.3。

2 任务 1:demo 输出与 $body_{pose}$

2.1 任务要求

在“smplx_demo.py”的基础上, 自定义代码中的随机种子 `torch.manual_seed(seed)`、自定义命令行中的输入参数 `gender` (“neutral”、“male”、“female”); 将 `body_pose` 初始设置为至少三个固定值, 运行代码并保存每个人体模型的参数与重建后的模型。

结合理论课第 14 讲, 分析代码中的 `smplx` 模型生成逻辑, 在实验文档中回答并解释:

(1) “smplx_demo.py”中 `body_pose` 被初始化为为什么? 这在模型中体现为什么姿势? `body_pose` 代表的含义是什么?

(2) 将 `body_pose` 初始化为不同的固定值 (如 0.1、-0.1、0.2), 不同的初始值对模型姿势有什么影响? 根据结果概括分析不同关节点的变化, 并给出由 `body_pose` 得到最终每个关节点坐标的公式。

(3) 同样是重建模型, 实验七与实验九重建的区别是什么? 从输入、输出、重建速度、原理、是否需要多轮优化等方面分析。

2.2 代码修改

设置初始 `body_pose` 的代码如下。后续调整了输出与可视化的顺序以确保可以输出文件。

```
torch.manual_seed(2333) # 设置随机种子以保证结果可复现
betas, expression = None, None # 初始化形状和表情参数为 None
if sample_shape: # 如果需要随机采样形状
    betas = torch.randn([1, model.num_betas],
                        dtype=torch.float32) # 随机生成形状参数
    body_pose = torch.full([1, 21, 3], 0.1,
                          dtype=torch.float32) # 初始化为固定值0.5
if sample_expression: # 如果需要随机采样表情
    expression = torch.randn(
        [1, model.num_expression_coeffs],
        dtype=torch.float32) # 随机生成表情参数
```

我们使用以下代码在终端中进行运行：

```
python smplx_demo.py --model-folder models  
--plot-joints=True --gender="neutral"
```

2.3 运行结果

以下分别是 body_pose 这个地方中的参数分别为 0、0.1、0.2、-0.1 四种所输出的模型的输出结果截图。

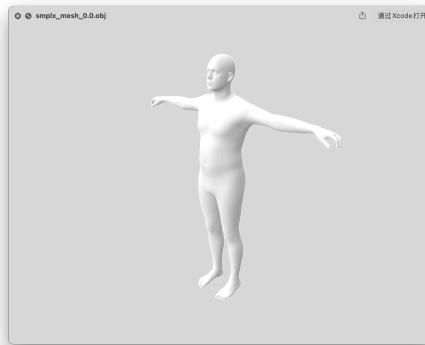


图 1: 实验 1 结果: 0.0

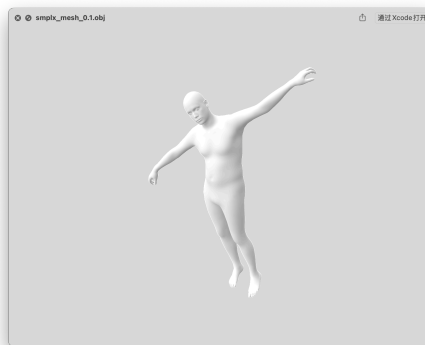


图 2: 实验 1 结果: 0.1

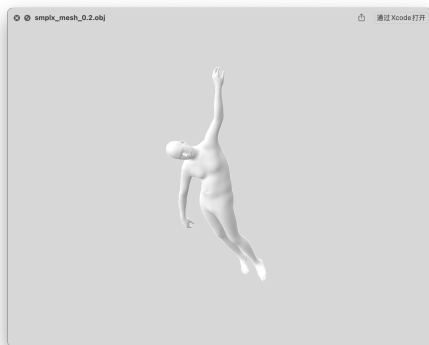


图 3: 实验 1 结果: 0.2

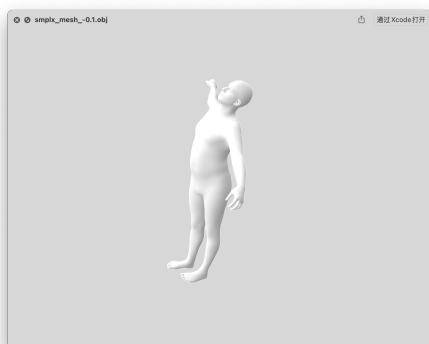


图 4: 实验 1 结果: -0.1

我们可以发现，该参数会影响输出姿态。

2.4 body_pose 含义

body_pose 被初始化为: `body_pose = torch.full([1, 21, 3], 0.0, dtype=torch.float32)`。

即: body_pose 的所有旋转轴角值初始为 0，每个局部关节的姿态是无旋转。这意味着所有局部关节保持“默认姿态”，模型呈现一个双臂横平、直立站立的人体。即 T-Pose。

body_pose 是一个 [1,21,3] 的张量，表示 SMPL-X 模型中 21 个身体关节的旋转姿态。它描述的是每个关节相对于其父关节的旋转，不包含全局朝向。

2.5 不同初始化值

如图 1、图 2、图 3、图 4 所示，当初始化分别为 0、0.1、0.2、-0.1 时，最终模型呈现不一样的姿态。当初始为小正值时，关节会产生轻微弯曲，手臂、腿部略微向前或外侧摆动。当取负数时，会出现与正值相反方向的摆动；当值增大时，摆动更明显。

针对从 `body_pose` 生成顶点坐标的公式的问题, ChatGPT 给出的回答是: $\mathbf{J}_i = (\prod_{k \in \text{path}(i)} T_k) \cdot \mathbf{J}_{\text{rest}, i}$ 。我查询 SMPL 原论文唯一给出的与顶点相关的公式如下: $J(\beta; J, \bar{T}, S) = J(T + BS(\beta; S))$ 。论文中以后相关内容如下: “其中 J 是一个将静止顶点位置转换为关节位置的矩阵。我们通过大量不同个体和姿态的训练数据, 学习得到这个回归矩阵 J , 作为我们整体模型学习的一部分。”

2.6 与可微渲染

实验七是基于可微渲染的, 其核心是从球不断优化。SMPL 则是基于参数的, 直接生成结果。将比较结果列表如下。

比较维度	实验七: 基于可微渲染	实验九: 基于 SMPL-X
输入类型	RGB 图像、初始网格	随机生成的形状和姿态参数
输出结果	优化后的 3D 网格	参数化人体模型 mesh
核心原理	可微渲染最小化渲染误差	代数原理
优化方式	梯度下降等逐步拟合	可直接设定参数无需梯度优化
重建速度	慢, 需多轮渲染、反向传播	快, 一次前向传播即可
是否需要多轮优化	是 (通常上百轮)	否 (直接生成)

表 1: 实验七与实验九的对比

3 任务 2:Blender 调整

3.1 任务要求

在 Blender 中增加 `smplx` 插件, 并导入任务一得到的人体参数 (初始值不能是 0.0), 手动调整关节点, 使其姿势重新接近标准姿态, 通过 Blender 保存调整后模型的参数与模结合理论课第 14 讲与 `smplx` 项目的介绍部分 (github), 结合 Blender 手动调整时关节点的位置变化对蒙皮的影响, 在实验文档中回答并解释:

(1) 蒙皮的思想是什么? SMPL-X 模型中使用的蒙皮方法是 LBS、DQS、... 中的哪种?

(2) 结合相关论文的视频介绍 (参考链接已在开头给出) 与理论课的内容, 给出实验所用蒙皮方法的公式, 并分析 Blender 可视化中关节点的位置、方向对蒙皮结果的影响。

(3) 如果想要在 SMPL-X 模型的基础上, 编写代码以自动调整为合理姿态, 可以通过限制什么来优化? 根据你的理解回答即可。

3.2 运行结果

本部分主要依靠 Blender 进行调整。按照 YouTube 教程，将任务 1 中生成的以 0.1 为初始值的模型参数导入 Blender 中 SMPL-X 插件，并通过对比将每个关节点进行逐一调整。这里需要强调的是，由于父关节会影响子关节，所以应当先调整父关节，依次进行。如图，我们将原来的模型（如图 5）一步一步变为标准情形（如图 6），其中如果我们对一个顶点进行太过离谱的调整后会得到如图 7。

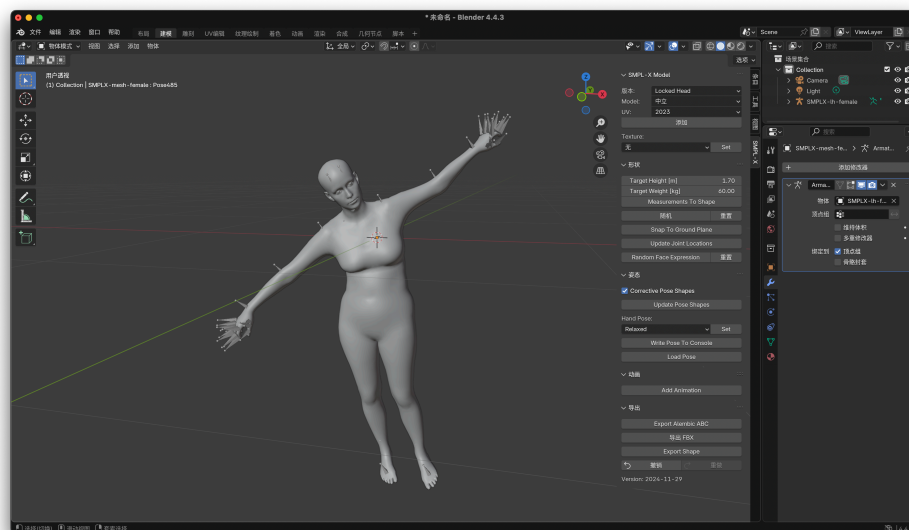


图 5: 实验 2 初始

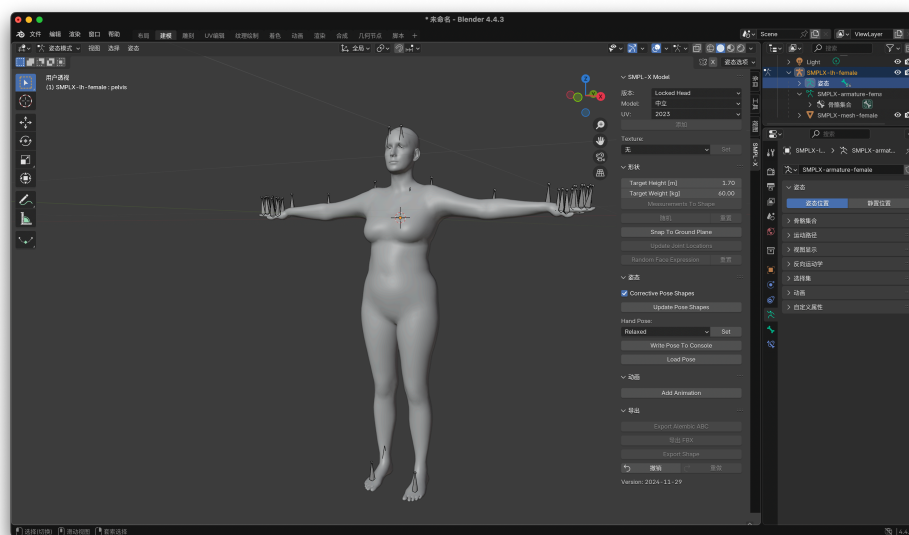


图 6: 实验 2 结果

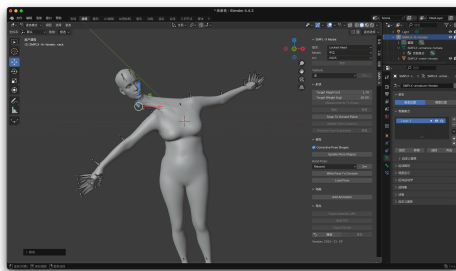


图 7: 实验 2 离谱

3.3 蒙皮的思想

蒙皮旨在将三维网格与骨架绑定，使得当骨骼发生变化时，网格表面能够实现相应的变形，产生自然的表面效果。其核心思想是：通过权重函数，将网格顶点的位置变形由一个或多个骨骼控制，网格顶点的位置是其绑定骨骼变换的加权组合。

SMPL-X 模型采用的是线性混合蒙皮（LBS）方法。

3.4 蒙皮公式

SMPL 所使用的蒙皮公式如下： $M(\beta, \theta) = W(T(\beta, \theta), J(\beta), \theta, \mathcal{W})$ 。其中： $M(\beta, \theta)$ 是最终的网格顶点位置； $T(\beta, \theta) = \bar{T} + B_s(\beta) + B_p(\theta)$ 是加上形状与姿态修正的 mesh； $J(\beta)$ 是关节位置； $\mathcal{W}$ ：蒙皮权重矩阵，维度为 $N \times K$ ， N 为顶点数， K 为关节数； $W(\cdot)$ 是线性混合蒙皮函数。

在 Blender 中手动旋转或移动关节时，模型表面会随之发生变形。旋转时，会控制关节相连顶点的方向，如旋转手臂根部关节会引起手臂网格的弯曲。位置变换会改变骨骼位置。

3.5 合理限制

加入物理限制，如关节角度限制：通过约束每个关节旋转的合法范围（例如肘关节不可反向），防止不自然姿态出现。再比如要求骨骼长度不变避免极端异常，如图 7。

4 任务 3: VPoser

4.1 任务要求

对 “smplx_demo.py”、“smplx_vposer.py”，设置相同的参数（随机种子、gender），body_pose 均由 torch.randn 随机生成，运行代码并保存两个人体模型参数与重建后的模型。

结合理论课第 14 讲与 human_body_prior 项目的介绍部分、训练部分（github），结合代码与优化结果，在实验文档中回答并解释：

- (1) VPoser 网络的输入和输出分别是什么？它在人体模型重建中起到什么作用？
- (2) VPoser 所用的深度学习网络是哪种？该网络的特点是什么？
- (3) 在 human_body_prior 项目中的高级 IK 功能部分中，介绍了 VPoser 的进阶使用。以动态捕捉的应用为例，该类任务的关键点在于什么？根据你的理解回答即可。

4.2 代码修改与结果展示

4.2.1 demo

```
torch.manual_seed(2333) # 设置随机种子以保证结果可复现
betas, expression = None, None # 初始化形状和表情参数为 None
if sample_shape: # 如果需要随机采样形状
    betas = torch.randn([1, model.num_betas],
                        dtype=torch.float32) # 随机生成形状参数
    body_pose = torch.randn([1, 21, 3],
                           dtype=torch.float32)
if sample_expression: # 如果需要随机采样表情
    expression = torch.randn(
        [1, model.num_expression_coeffs],
        dtype=torch.float32) # 随机生成表情参数
```

通过随机进行初始化。

我们使用以下代码在终端中进行运行：

```
python smplx_demo.py --model-folder models
--plot-joints=True --gender="neutral"
```

得到结果如图 8。

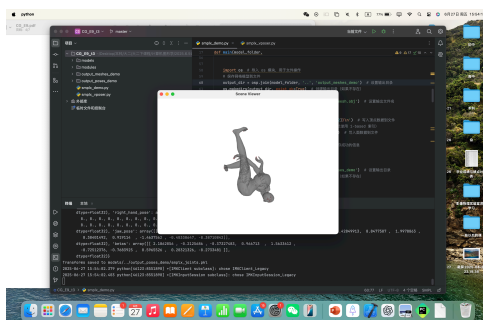


图 8: 实验 3demo 结果

4.2.2 VPoser

```

torch.manual_seed(2333) # 设置随机种子以保证结果可复现
betas, expression = None, None # 初始化形状和表情参数为 None
if sample_shape: # 如果需要随机采样形状
    betas = torch.randn([1, model.num_betas],
                        dtype=torch.float32) # 随机生成形状参数
    body_pose = torch.randn([1, 21, 3],
                           dtype=torch.float32)
if sample_expression: # 如果需要随机采样表情
    expression = torch.randn(
        [1, model.num_expression_coeffs],
        dtype=torch.float32) # 随机生成表情参数
# VPoser优化刚刚随机生成的body_pose
expr_dir = osp.join(model_folder, 'vposer', 'V02_05')
# 'TRAINED_MODEL_DIRECTORY' in this directory
the trained model along with the model code exist
print(f'Loading VPoser from {expr_dir}')
# 打印加载 VPoser 的目录
from human_body_prior.tools.model_loader
import load_model
from human_body_prior.models.vposer_model
import VPoser
vp, ps = load_model(expr_dir, model_code=VPoser,
                    remove_words_in_model_weights='vp_model.', disable_grad=True)

body_pose = body_pose.reshape(1, -1)
body_poZ = vp.encode(body_pose).mean
body_pose_rec = vp.decode(body_poZ)['pose_body']
.contiguous().view(-1, 63)

```

随机初始化后进行 VPoser 优化。

我们使用以下代码在终端中进行运行：

```

python smplx_vposer.py --model-folder models
--plot-joints=True --gender="neutral"

```

得到结果如图 9。

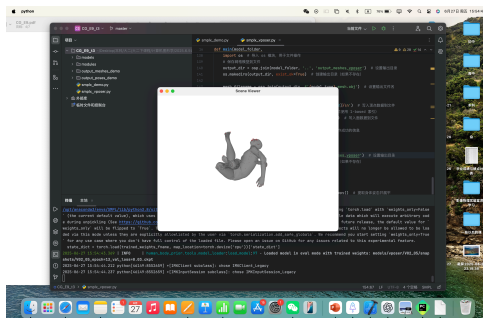


图 9: 实验 3vposer 结果

4.3 输入输出与作用

输入：VPoser 网络的输入是一个向量，表示人体姿态的表示。在本代码中，输入为 `body_pose`。

输出：输出是 SMPL-X 模型中所需的 `body_pose` 参数。姿态参数最终被用于人体网格的重建。

作用：VPoser 在人体模型重建中扮演姿态先验的角色。人体自然姿态的空间分布是高度非线性的，`torch.randn` 初始化往往会产生非人类、非自然的姿态。VPoser 通过在大规模真实人体姿态数据集上训练，学习到一个映射函数： $\mathbf{z} \in \mathbb{R}^d \rightarrow \mathbf{p} \in \mathbb{R}^{63}$ 。该映射能从潜在空间 $\mathbf{z}$ 中解码出合理的人体姿态 $\mathbf{p}$ 。

4.4 深度学习网络

VPoser 是基于变分自编码器（VAE）的。

VAE 网络的特点：生成能力强：VAE 通过学习数据的分布，可以从低维空间采样生成高质量的自然姿态；潜在空间连续：保证了从潜在向量连续插值也能生成连续、合理的姿态；正则化能力：在人体姿态优化过程中，提供了“合理性先验”，避免不自然的动作。

4.5 进阶使用

在 Human Body Prior 项目的高级 IK 应用中，VPoser 不仅作为一个姿态生成器，还起到了约束解空间、提升姿态自然性与稳定性的作用。以动态捕捉为例，该任务的关键点主要包括以下几个方面：

时序一致性：动态动作捕捉涉及连续帧的估计，因此姿态估计应具备良好的时间一致性。VPoser 的潜在空间具有流形结构，支持在低维空间中对姿态变化进行平滑插值，显著提升序列姿态的一致性和自然过渡性。

自然性先验：由于真实捕捉数据经常存在遮挡或关键点检测误差，使用 VPoser 作为姿态生成器可以显著减少这些噪声影响，避免非自然或解剖学上不可行的解。

优化效率：IK 问题中目标是最小化关键点误差。直接在高维空间（如 63 维关节空间）优化代价高、易陷入局部最优。而 VPoser 将优化空间降低为潜在维度（如 32 维），提高了优化速度和精度，且更稳定。

5 结束语

终于结束了~ 现在是 6 月 28 日 1:44……熬夜不是什么好习惯，但考完试的白天真的懒得动脑。很荣幸能够成为《计算机图形学》换老师后的第一届学生，虽然课程的内容、过程都跌跌撞撞，但在不断的交流-调整的循环之中，我们也在陪伴着这门课不断变好。

现在回头看，第一次做实验时的崩溃犹在眼前，但要是让我现在来做的话或许半天不到就能做完吧，但是却要做两天，在进步~ 就这样，晚安！